



Concorrência em Bancos Relacionais

Professor: Moisés Sousa

Objetivos da Aula

- Compreender concorrência em bancos relacionais.
- Entender transações e ACID.
- Aprender *locks* e *deadlocks*.
- Analisar problemas reais de concorrência.

Concorrência em Bancos Relacionais

- Transações.
- Locks.
- Deadlocks.
- ACID.
- Controle de concorrência.

O que é concorrência?

- Execução simultânea de operações no banco.
- Múltiplos usuários acessando os mesmos dados.
- Comum em sistemas bancários, e-commerce e ERPs.

Problema da Concorrência

- Dois usuários tentando sacar simultaneamente.
- Ambos leem o mesmo saldo.
- Atualizações conflitantes podem gerar inconsistência.

Exemplo sem Controle

- Saldo inicial: 1000
- Usuário A saca 700
- Usuário B saca 500
- Resultado incorreto: saldo final inconsistente

O que é uma Transação?

- Conjunto de operações tratadas como unidade lógica.
- Tudo acontece ou nada acontece.
- Garantia de consistência.

Estrutura de Transação

- START TRANSACTION.
- UPDATE.
- COMMIT.
- ROLLBACK.

Estrutura de Transação

- START TRANSACTION.
- UPDATE.
- COMMIT.
- ROLLBACK.



COMMIT

- Confirma permanentemente as alterações.
- Libera *locks*.
- Finaliza a transação.



ROLLBACK

- Desfaz alterações da transação.
- Recupera estado anterior.
- Mantém consistência dos dados.

Como a Transação Resolve o Problema

- O banco controla acesso simultâneo.
- Uso de *locks*.
- Uma transação espera a outra terminar.



SELECT FOR UPDATE

- Bloqueia a linha selecionada.
- Impede alterações simultâneas.
- Outras transações aguardam.

Fluxo da Solução

- Transação A bloqueia registro.
- Transação B espera.
- A realiza COMMIT.
- B lê valor atualizado.



ACID

- Atomicidade.
- Consistência.
- Isolamento.
- Durabilidade.



Atomicidade

- Tudo ou nada.
- Se falhar, tudo é desfeito.



Consistência

- Banco permanece válido.
- Regras de integridade preservadas.



Isolamento

- Transações não interferem incorretamente.
- Controle de concorrência.



Durabilidade

- Após COMMIT os dados persistem.
- Mesmo após falhas ou quedas.

Problemas de Concorrência

- *Dirty Read* (Leu algo que talvez nunca exista).
- *Non-repeatable Read* (A mesma linha mudou).
- *Phantom Read* (Novas linhas apareceram).



Níveis de Isolamento

- READ UNCOMMITTED
- READ COMMITTED
- REPEATABLE READ
- SERIALIZABLE

Níveis de Isolamento

Isolation Level	Dirty	Non-repeatable	Phantom
READ UNCOMMITTED	✗	✗	✗
READ COMMITTED	✓	✗	✗
REPEATABLE READ	✓	✓	✗
SERIALIZABLE	✓	✓	✓

Locks

- Mecanismos de bloqueio.
- *Shared Lock*.
 - todo mundo pode ler,
 - ninguém pode alterar.
- *Exclusive Lock*.
 - ninguém mais pode mexer naquele documento.

Deadlock

- Duas transações aguardam indefinidamente.
- Ciclo de espera.
- Banco detecta e cancela uma delas.

Como Evitar Deadlocks

- Acessar tabelas na mesma ordem.
- Transações curtas.
- Índices corretos.
- Reduzir tempo de *lock*.



Aplicações Reais

- Transferências bancárias.
- Compras simultâneas.
- Sistemas acadêmicos.
- Controle de estoque.

Conclusão

- Concorrência é inevitável.
- Transações garantem consistência.
- Locks e ACID protegem os dados.
- SGBDs controlam acesso simultâneo.

Teste Rápido - 05

Uma escola possui uma bolsa de pesquisa com saldo inicial de R\$ 1000,00.

Dois alunos tentarão realizar saques simultaneamente dessa mesma bolsa:

- Aluno A: saque de R\$ 700,00;
- Aluno B: saque de R\$ 500,00.

A atividade deverá ser realizada em dois cenários:

- Sem utilização de transações;
- Com utilização de transações e `SELECT ... FOR UPDATE`.

Teste Rápido - 05

Ao final, compare os resultados obtidos nos dois casos e analise:

- o saldo final da bolsa;
- a ocorrência de inconsistências;
- o comportamento do banco diante de acessos simultâneos;
- o papel das transações no controle de concorrência.